Voice Orchestrator — Identity API Guide

Login, sign-up, and how the SmartHome app gets user_ref and home_id at startup (no more hardcoded values) · for Aaron · July 2026 · v1.2

The questions this answers

“How does the app know the values for user_ref and home_id on startup?” — **it asks the server.** The user logs in once and the server returns their user_ref, home_id(s), and — as requested — their user ID and email for feedback reports. Cache these at startup and use them everywhere the app currently hardcodes "scott_mobile" / "scott_home".

“Looks like I'm going to need to add a Login Screen. Do we have a method for sign-up?” — **Yes: POST /auth/signup** (section 0 below). New accounts start pending; HomeAdapt approves them from the admin portal and attaches the home, then the user can log in. So the app needs a login screen, a “Create account” screen, and a change-password screen — all four endpoints are LIVE in production today.

Base URL & auth model

Item	Value
API base (BASE)	https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
Login	POST $BASE/auth/login (no auth header needed)
Everything else	Authorization: Bearer <token from login>
Token lifetime	30 days; on 401 re-login
Old static keys	sk_ios_… etc. still work unchanged during transition

App startup flow

```
1. Have a stored token?
     NO  -> show login screen -> POST /auth/login -> store token
     YES -> GET /me with the token
              200 -> refresh cached identity
              401 -> token expired -> show login screen
2. Cache from the response:  user_ref, default_home_id, user_id, email
3. Use user_ref + default_home_id in every favorites / enrollments /
   search / trigger call (exactly where "scott_mobile" / "scott_home"
   are hardcoded today).
```

0 · POST /auth/signup — “Create account”

New accounts start **pending**: the user can register, but cannot log in until HomeAdapt activates the account and attaches their home (each home needs a hardware install, so onboarding stays admin-controlled — signups appear in the admin portal with a one-click Activate). No email verification in v1 — it ships together with self-service password reset later.

```
curl -s -X POST "$BASE/auth/signup" \
  -H "Content-Type: application/json" \
  -d '{"email": "new@example.com", "password": "min-8-chars", \
      "full_name": "New User"}'
```

Status	Meaning	App behavior
201 pending_approval	Account created, awaiting activation	Show the returned **message**, route to login screen
409 EMAIL_EXISTS	Email already registered	Offer “Log in instead”
400 VALIDATION	Bad email / weak password / missing name	Inline validation
429 RATE_LIMITED	5 signups per 15 min per IP	“Try again later”

Until activated, login with the correct password returns 403 PENDING_APPROVAL — show “your account is awaiting activation.” Once activated with no home attached yet, /me returns homes: [] and default_home_id: null — show an “awaiting home setup” state.

1 · POST /auth/login

Accepts **email** or **username** plus password. Returns the token AND the full identity payload, so startup is one call.

```
curl -s -X POST "$BASE/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email": "scott@example.com", "password": "********"}'
```

Response 200:

```
{
  "token":           "eyJhbGciOiJIUzI1NiIs...",
  "token_type":      "Bearer",
  "expires_in":      2592000,
  "user_ref":        "scott_mobile",     <- use in every user_ref field
  "user_id":         "scott_mobile",     <- for feedback reports
  "username":        "scottmeyers",
  "email":           "smeyersne@gmail.com",  <- for feedback reports
  "full_name":       "scott meyers",
  "homes":           [ { "home_id": "scott_home", "name": "scott_home" } ],
  "default_home_id": "scott_home"        <- use in every home_id field
}
```

Status	Meaning	App behavior
200	Logged in	Store token, cache identity fields
400 VALIDATION	Missing/invalid email or password	Show inline validation
401 UNAUTHORIZED	Wrong credentials (one generic message)	Show “Invalid credentials”
403 PENDING_APPROVAL	Account not activated yet	Show “awaiting activation”
429 RATE_LIMITED	5 failed attempts → 15-min lock	Show “try again later”; do NOT auto-retry

2 · GET /me

Same identity payload (minus token fields) — call on every app start while a token is stored.

```
curl -s "$BASE/me" -H "Authorization: Bearer $TOKEN"
```

401 → token expired or invalid → show the login screen. Note: /me only works with a login token — the old static platform keys carry no user identity and are rejected here on purpose.

3 · POST /auth/change-password

For the in-app “Change password” screen. Requires the login token AND the current password (a stolen token alone cannot change it).

```
curl -s -X POST "$BASE/auth/change-password" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"current_password": "old-one", "new_password": "new-min-8-chars"}'
```

Status	Meaning
204	Changed. Current token stays valid — no forced re-login.
403 FORBIDDEN	Current password is incorrect
400 VALIDATION	New password outside 8–256 characters
429 RATE_LIMITED	5 wrong current-password attempts → 15-min lock
401 UNAUTHORIZED	Missing/expired token → re-login

Forgotten password: no self-service reset yet — Karthi resets it via the admin API and hands the user a new one.

4 · Using the token on existing endpoints

The login token is a drop-in replacement for the static key on ALL existing endpoints — identical header shape:

```
Authorization: Bearer <token>
```

Because the server now knows who is calling, two new 403 rules apply to token callers (never to static-key callers):

Rule	Result
Request carries a **user_ref** (query, form, or JSON) that is not the logged-in user's	403 FORBIDDEN
Request carries a **home_id** the logged-in user does not own	403 FORBIDDEN

Worked example — add a favorite, nothing hardcoded:

```
# values cached from login/me:
USER=<user_ref from /me>     HOME=<default_home_id from /me>

curl -s -X POST "$BASE/favorites" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d "{\"user_ref\":\"$USER\",\"home_id\":\"$HOME\", \
      \"device_id\":\"6b86cd8c539ad69b193a8ff2acbf3b4e\"}"
```

5 · User ID / email for feedback (as requested)

Both login and /me return **user_id** and **email**. Attach them to every feedback submission so reports are traceable to a person:

```
{
  "feedback":  "...user's text...",
  "user_id":   "<user_id from /me>",     // e.g. "scott_mobile"
  "email":     "<email from /me>",       // e.g. "smeyersne@gmail.com"
  "user_ref":  "<user_ref from /me>",
  "home_id":   "<default_home_id from /me>"
}
```

Integration checklist

#	Item
0	Sign-up screen → POST /auth/signup → “pending activation” message
1	Remove hardcoded “scott_mobile” / “scott_home” constants
2	Login screen → POST /auth/login → store token securely (Keychain/Keystore)
3	On startup with token → GET /me → cache user_ref, default_home_id, user_id, email
4	All API calls use the token as bearer + cached identity values
5	Global 401 handler → clear token → login screen; 429 → “try later” message
6	Change-password screen → POST /auth/change-password
7	Feedback reports include user_id + email from /me

Status: all endpoints above are deployed and verified in production (voiceorchestrator.homeadapt.us). Scott's account is provisioned; credentials come from Karthi. Full endpoint reference: docs/mobile_handoff.md §2.1.